

Azure Databricks ETL Project

E-commerce/Retail Data Platform — Complete Self-Guided Build

Your environment: Trial (Premium - 14-Days Free DBUs) · Region: Central India · Admin access confirmed

Scenario: customers, products, orders, order_items, payments, shipments for an online retail storefront.

How to use this document

- This covers all 17 stages (Task 0 - Task 16) in one place so you can work at your own pace and explore the UI freely, instead of waiting on a reply after each step.
- Do the tasks in order — each one builds on tables, files, or notebooks created in an earlier task.
- Every task uses the same fixed structure: Difficulty, What You'll Learn, Architecture, Why, Prerequisites, Cost Warning, UI Steps, Code, Expected Output, Verify, Common Errors, Cleanup, Mini Exercise.
- When something on your screen doesn't match a described button or menu exactly, look for the closest equivalent — Databricks updates its UI often. Come back to chat any time you get a genuine error and want it debugged.
- Keep this rule for the whole project: prefer serverless compute, use single-node/smallest classic clusters when serverless isn't offered, set auto-termination to 10-15 minutes on any classic cluster, and delete anything you created purely for a mini exercise.

TASK 0 — Inspect Your Existing Environment

Difficulty: Very Easy

Already covered: your workspace is on **Trial (Premium - 14-Days Free DBUs)**, in **Central India**, and you have admin access. Premium trial workspaces have Unity Catalog enabled by default, so Task 3 onward can assume it's available. Before Task 1, spend two minutes confirming the rest for yourself:

- 1 Open **Catalog** in the left sidebar — confirm you can see at least one catalog (often named workspace).
- 2 Open **Compute** → **Create compute** — note whether a **Serverless** option appears; don't click Create.
- 3 Open your name/profile icon → **Settings** — confirm the Admin Console section is visible to you (you already confirmed this).
- 4 In the Azure Portal, check your own resource group for any existing Storage Account — note its name if one exists, for reuse in Task 5.

Nothing here costs money — it's UI browsing only. If any of these look different from what's described, note it and keep going; Task 1 doesn't depend on the answers, but Task 3 and Task 5 do.

TASK 1 — Direct Data Loading

Difficulty: Very Easy

What You'll Learn

- Creating a notebook and attaching compute
- Uploading a file into a Unity Catalog Volume
- `spark.read.csv` to load a DataFrame, and `df.write.format("delta")` to save it

Architecture

```
customers.csv -> Volume -> spark.read.csv() -> DataFrame
                        |
                        v
                df.write.format("delta").saveAsTable()
                        |
                        v
workspace.default.customers (Delta table)
```

Why

This is the smallest possible end-to-end loop in Databricks: file in, DataFrame, table out, query back. Every later task is a bigger version of this same loop.

Cost Warning

Only the compute you attach can cost anything. Prefer **Serverless** if offered; otherwise a single-node classic cluster with the smallest node type and **10-15 minute auto-termination**.

Setup

- 1 Save the CSV block below as `customers.csv` on your computer.
- 2 Left sidebar → **+ New** → **Notebook**. Rename it `Task1_DirectLoad`, language Python.
- 3 Attach compute: click the compute dropdown at the top → pick **Serverless** if listed, otherwise **Create new** → smallest single-node cluster → set **Auto Termination** to 15 min.
- 4 Left sidebar → **+ New** → **Add or upload data** → **Upload files to a volume** → pick your catalog (e.g. `workspace`), schema `default`, create a volume named `raw_files` if none exists → upload the CSV → copy the resulting path.

customers.csv

```
customer_id,customer_name,email,city,state,signup_date
1001,Aarav Shah,aarav.shah@example.com,Ahmedabad,Gujarat,2025-01-14
1002,Priya Nair,priya.nair@example.com,Bengaluru,Karnataka,2025-01-20
1003,Rohan Mehta,rohan.mehta@example.com,Pune,Maharashtra,2025-02-02
1004,Sneha Iyer,sneha.iyer@example.com,Chennai,Tamil Nadu,2025-02-11
1005,Karan Desai,karan.desai@example.com,Ahmedabad,Gujarat,2025-02-18
1006,Anjali Rao,anjali.rao@example.com,Hyderabad,Telangana,2025-03-01
1007,Vikram Joshi,vikram.joshi@example.com,Surat,Gujarat,2025-03-09
1008,Neha Kulkarni,neha.kulkarni@example.com,Pune,Maharashtra,2025-03-15
1009,Arjun Reddy,arjun.reddy@example.com,Hyderabad,Telangana,2025-03-22
1010,Divya Menon,divya.menon@example.com,Kochi,Kerala,2025-04-02
```

Cell 1 — read

```
csv_path = "/Volumes/workspace/default/raw_files/customers.csv" # use YOUR path
```

```
df = spark.read.csv(csv_path, header=True, inferSchema=True)
```

Cell 2 — inspect

```
df.printSchema()
df.show()
print(f"Row count: {df.count()}")
```

Cell 3 — write as Delta

```
df.write.format("delta").mode("overwrite").saveAsTable("workspace.default.customers")
```

Cell 4 — query back (SQL)

```
%sql  
SELECT * FROM workspace.default.customers;
```

Expected Output

- 6 columns, 10 rows; row count prints as 10.
- The SQL query returns the same 10 rows from the saved table.

Verify

Catalog (sidebar) → workspace → default → Tables → confirm customers is listed with a Sample Data preview.

Common Errors

- **Path does not exist** — re-copy the exact path the upload UI showed you.
- **Catalog 'workspace' not found** — substitute your actual catalog name everywhere.
- Cell hangs — compute is still starting; wait for a green/running state in the compute dropdown.

Cleanup

Auto-termination handles the cluster. Keep the table and volume file — Task 2 builds on them.

Mini Exercise

Write one line of PySpark that counts how many customers are from "Gujarat" (filter on state). Don't ask for the answer before trying it.

TASK 2 — Basic Data Transformation

Difficulty: Easy

What You'll Learn

- select, filter, withColumn, cast, withColumnRenamed, drop
- Null handling (na.fill / na.drop / filter with isNotNull)
- dropDuplicates
- groupBy + agg aggregations
- joins across three tables

Architecture

```
customers (Task 1) + products.csv + orders.csv
      |           |           |
      v           v           v
    spark.read  spark.read  spark.read  -> 3 DataFrames
      |
      v
    select / filter / cast / groupBy / join
      |
      v
workspace.default.products, workspace.default.orders
```

Why

Raw columns are rarely the right type, name, or shape for analysis. This task is the transformation vocabulary you'll reuse in every Bronze-to-Silver step from Task 4 onward.

Cost Warning

Same compute rules as Task 1 — nothing new to spin up.

Setup

- 1 Save the two CSV blocks below as `products.csv` and `orders.csv`.
- 2 Upload both the same way as Task 1 (New → Add or upload data → Upload files to a volume → same `raw_files` volume).
- 3 New notebook: `Task2_Transformations`, attach the same compute.

products.csv

```
product_id,product_name,category,price
2001,Wireless Mouse,Electronics,699.00
2002,Bluetooth Speaker,Electronics,1899.00
2003,Cotton T-Shirt,Apparel,499.00
2004,Running Shoes,Apparel,2499.00
2005,Non-stick Pan,Home,1299.00
2006,Yoga Mat,Fitness,799.00
2007,Desk Lamp,Home,999.00
2008,Backpack,Apparel,1599.00
2009,Water Bottle,Fitness,349.00
2010,Phone Charger,Electronics,449.00
```

orders.csv

```
order_id,customer_id,product_id,quantity,order_date,status
5001,1001,2001,1,2025-04-01,COMPLETED
5002,1002,2004,2,2025-04-01,COMPLETED
5003,1003,2005,1,2025-04-02,COMPLETED
5004,1001,2010,3,2025-04-03,COMPLETED
5005,1004,2003,1,2025-04-03,CANCELLED
```

```
5006,1005,2002,1,2025-04-04,COMPLETED
5007,1006,2008,1,2025-04-05,COMPLETED
5008,1007,2006,2,2025-04-05,COMPLETED
5009,1008,2009,4,2025-04-06,COMPLETED
5010,1009,2007,1,2025-04-06,COMPLETED
5011,1010,2001,2,2025-04-07,COMPLETED
5012,1002,2005,1,2025-04-07,COMPLETED
5013,1003,2008,1,2025-04-08,CANCELLED
5014,1004,2010,2,2025-04-08,COMPLETED
5015,1001,2004,1,2025-04-09,COMPLETED
```

Cell 1 — load all three

```
products = spark.read.csv("/Volumes/workspace/default/raw_files/products.csv", header=True, inferSchema=True)
orders    = spark.read.csv("/Volumes/workspace/default/raw_files/orders.csv",   header=True, inferSchema=True)
customers = spark.table("workspace.default.customers") # reuse Task 1's table
```

Cell 2 — select / filter / withColumn / cast / rename / drop

```
from pyspark.sql.functions import col

orders_clean = (
    orders
    .select("order_id", "customer_id", "product_id", "quantity", "order_date", "status")
    .withColumn("quantity", col("quantity").cast("int"))
    .withColumnRenamed("order_date", "order_dt")
    .filter(col("status") == "COMPLETED") # drop cancelled orders
)
orders_clean.show()
```

Cell 3 — null handling and duplicates

```
orders_no_nulls = orders_clean.filter(col("quantity").isNotNull())
orders_dedup    = orders_no_nulls.dropDuplicates(["order_id"])
print(orders_dedup.count())
```

Cell 4 — groupBy / agg

```
from pyspark.sql.functions import sum as _sum, count as _count

revenue_by_status = (
    orders.groupBy("status")
    .agg(_count("order_id").alias("num_orders"))
)
revenue_by_status.show()
```

Cell 5 — joins

```
order_details = (
    orders_dedup
    .join(products, on="product_id", how="inner")
    .join(customers, on="customer_id", how="inner")
    .select("order_id", "customer_name", "product_name", "category", "quantity", "price")
)
order_details.show()
```

Cell 6 — save the new tables

```
products.write.format("delta").mode("overwrite").saveAsTable("workspace.default.products")
orders_dedup.write.format("delta").mode("overwrite").saveAsTable("workspace.default.orders")
```

Expected Output

- `orders_clean`: 13 rows (2 CANCELLED rows removed).
- `order_details`: joined rows showing `customer_name`, `product_name`, `category`, `quantity`, `price` together.

Verify

Catalog Explorer → confirm `products` and `orders` tables now exist alongside `customers`.

Common Errors

- **Column not found after join** — both sides had a column with the same name; qualify with the DataFrame alias or drop the duplicate column first.

- **cast() silently produces null** — the source value wasn't a valid number/date for that type; check with `.filter(col(x).isNull())` before and after casting.

Cleanup

Nothing to delete — these tables feed Task 3 and Task 4.

Mini Exercise

Using `order_details`, compute total revenue (`quantity * price`) per category with `groupBy + agg`.

TASK 3 — Unity Catalog

Difficulty: Easy-Medium

What You'll Learn

- Catalog -> Schema -> Table hierarchy
- Managed vs external tables (conceptually)
- Creating your own catalog and schemas

Architecture

```
ecommerce_dev      (catalog)
|-- bronze          (schema)
|   |-- customers, products, orders
|-- silver          (schema, built in Task 4)
|-- gold            (schema, built in Task 4)
```

Why

A **catalog** is the top-level container in Unity Catalog; a **schema** (a.k.a. database) sits inside it; a **table** sits inside a schema — giving every table a three-part name `catalog.schema.table`. This replaces the `workspace.default` layout you used in Tasks 1-2 with something that actually matches how a real project is organized.

Before Starting

Whether you can create a new catalog depends on your metastore permissions. As an admin on a trial workspace you almost certainly can — if the `CREATE CATALOG` statement below fails with a permission error, use an existing catalog instead and just create the three schemas inside it.

Cost Warning

Creating catalogs/schemas is free — it's metadata only. Only the compute running the SQL costs anything, same as before.

Setup / UI Steps

- 1 New notebook `Task3_UnityCatalog`, SQL as the default language (or use %sql cells).
- 2 Run the `CREATE` statements below.
- 3 Confirm in the sidebar under **Catalog** that `ecommerce_dev` now appears with three empty schemas.

Cell 1 — create catalog and schemas

```
%sql
CREATE CATALOG IF NOT EXISTS ecommerce_dev;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.bronze;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.silver;
CREATE SCHEMA IF NOT EXISTS ecommerce_dev.gold;
```

Cell 2 — move Task 1-2 tables into bronze (CTAS)

```
%sql
CREATE TABLE ecommerce_dev.bronze.customers AS SELECT * FROM workspace.default.customers;
CREATE TABLE ecommerce_dev.bronze.products AS SELECT * FROM workspace.default.products;
CREATE TABLE ecommerce_dev.bronze.orders AS SELECT * FROM workspace.default.orders;
```

Expected Output

Each `CREATE` statement returns no rows but succeeds; the tables now exist under `ecommerce_dev.bronze`.

Verify


```
%sql SELECT COUNT(*) FROM ecommerce_dev.bronze.orders; should return 13.
```

Common Errors

- **PERMISSION_DENIED creating catalog** — you're not a metastore admin; ask whoever is, or reuse an existing catalog and only create the three schemas.
- **Catalog already exists** — harmless with `IF NOT EXISTS`; safe to re-run.

Cleanup

Nothing to clean up. The old `workspace.default` copies can stay or be dropped — your choice; the project moves forward using `ecommerce_dev` from here on.

Mini Exercise

Run `SHOW TABLES IN ecommerce_dev.bronze;` and `DESCRIBE TABLE ecommerce_dev.bronze.orders;` and compare the output to what you'd expect.

TASK 4 — Medallion Architecture

Difficulty: Medium

What You'll Learn

- Why Bronze/Silver/Gold are separated the way they are
- Building a real Silver cleaning step
- Building real Gold business tables

Architecture

```
bronze.orders, bronze.customers, bronze.products    (raw, as-ingested)
      |
      v
silver.orders_clean    (typed, deduped, cancelled rows removed)
      |
      v
gold.daily_sales | gold.customer_summary | gold.product_performance
```

Why

Bronze keeps a faithful raw copy so you can always reprocess from scratch. **Silver** is where you fix types, remove duplicates/invalid rows, and standardize — it's still row-level, not aggregated. **Gold** is aggregated, business-facing, and directly answers questions like "what were yesterday's sales?"

Cost Warning

No new resources — same compute as before.

Cell 1 — silver.orders_clean

```
from pyspark.sql.functions import col, to_date

bronze_orders = spark.table("ecommerce_dev.bronze.orders")

orders_clean = (
    bronze_orders
    .filter(col("status") == "COMPLETED")
    .filter(col("quantity").isNotNull())
    .dropDuplicates(["order_id"])
    .withColumn("order_date", to_date(col("order_date")))
)
```

```
orders_clean.write.format("delta").mode("overwrite") \
    .saveAsTable("ecommerce_dev.silver.orders_clean")
```

Cell 2 — gold.daily_sales

```
from pyspark.sql.functions import sum as _sum, count as _count

orders_silver = spark.table("ecommerce_dev.silver.orders_clean")
products = spark.table("ecommerce_dev.bronze.products")

daily_sales = (
    orders_silver.join(products, "product_id")
    .withColumn("revenue", col("quantity") * col("price"))
    .groupBy("order_date")
    .agg(_sum("revenue").alias("total_revenue"), _count("order_id").alias("num_orders"))
    .orderBy("order_date")
)
daily_sales.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.gold.daily_sales")
```

Cell 3 — gold.customer_summary and gold.product_performance

```
customers = spark.table("ecommerce_dev.bronze.customers")

customer_summary = (
    orders_silver.join(products, "product_id")
```

```

    .withColumn("revenue", col("quantity") * col("price"))
    .join(customers, "customer_id")
    .groupBy("customer_id", "customer_name")
    .agg(_sum("revenue").alias("lifetime_spend"), _count("order_id").alias("num_orders"))
  )
customer_summary.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.gold.customer_summary")

product_performance = (
  orders_silver.join(products, "product_id")
  .withColumn("revenue", col("quantity") * col("price"))
  .groupBy("product_id", "product_name", "category")
  .agg(_sum("revenue").alias("total_revenue"), _count("order_id").alias("times_ordered"))
  .orderBy(col("total_revenue").desc())
)
product_performance.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.gold.product_performance")

```

Expected Output

- gold.daily_sales: one row per order_date with total_revenue and num_orders.
- gold.product_performance: ranked by total_revenue, descending.

Verify

Catalog Explorer → ecommerce_dev → gold → confirm all three tables exist and preview sample data on each.

Common Errors

- **Ambiguous column 'price'** after multiple joins — alias the DataFrames (e.g. `.alias("o")`) or select explicit columns before joining again.
- **revenue is null** — a join didn't match (missing product_id) — check with a left join and filter for nulls to find orphaned rows.

Cleanup

Nothing to delete. These Gold tables are the ones later tasks build a pipeline around.

Mini Exercise

Add a 4th Gold table, `gold.orders_by_state`, joining through customers to get revenue grouped by state.

TASK 5 — Azure Storage / Landing Area

Difficulty: Medium

What You'll Learn

- Storage Account, Container, ADLS Gen2, Access Connector, Storage Credential, External Location
- How Unity Catalog governs access to your own cloud storage

Architecture

```
Azure Storage Account (ADLS Gen2)
|-- container: landing
    |
    v
Access Connector for Azure Databricks (managed identity)
    |
    v
Unity Catalog Storage Credential -> External Location
    |
    v
Databricks reads/writes abfss://landing@<account>.dfs.core.windows.net/
```

Why

So far every file has lived inside a Unity Catalog Volume, which is convenient but hides the cloud storage underneath it. Real pipelines land files in a storage account you control, so Databricks needs to be explicitly granted access to it — that's what this task sets up.

Before Starting

Check whether you already have a Storage Account with Hierarchical Namespace (ADLS Gen2) enabled in your own resource group (from Task 0's Check 6). If yes, reuse it and just add a new container. If not, create one below.

Cost Warning

- A Storage Account itself is nearly free at this scale — billing is per GB stored and per operation (read/write/list), a few rupees for the amount of data in this project.
- The Access Connector for Azure Databricks is a free managed identity resource — it has no runtime cost by itself.
- Nothing here is deleted automatically — if you tear the project down later, delete the storage account explicitly.

UI Steps — Azure Portal

- 1 Search "Storage accounts" → **+ Create**. Subscription/resource group: your own. Name: something unique, e.g. stecomlearningXXXX. Region: **Central India** (match your Databricks workspace to avoid cross-region cost/latency).
- 2 Performance: Standard. Redundancy: LRS (cheapest, fine for learning).
- 3 On the **Advanced** tab, enable **Hierarchical namespace** (this is what makes it ADLS Gen2, required for Databricks external locations).
- 4 Review + Create.
- 5 Once created, open the storage account → **Containers** → **+ Container** → name it landing.
- 6 Search "Access Connector for Azure Databricks" → **+ Create** → same resource group/region → Create. This gives Databricks a managed identity Azure trusts.
- 7 Open the new Access Connector → copy its **Resource ID**.

- 8 Go back to your Storage Account → **Access Control (IAM)** → **+ Add role assignment** → role **Storage Blob Data Contributor** → assign to the Access Connector's managed identity (search its name).

UI Steps — Databricks

- 1 Sidebar → **Catalog** → **External Data** (or the small gear/plug icon) → **Credentials** tab → **Create credential** → type **Azure Managed Identity** → paste the Access Connector's Resource ID → name it `ecommerce_storage_cred`.
- 2 **External Locations** tab → **Create location** → URL: `abfss://landing@<youraccount>.dfs.core.windows.net/` → Credential: the one you just made → name it `ecommerce_landing`.
- 3 Test the connection from the same dialog — it should say the location is reachable.

Verify from a notebook

```
%sql  
LIST 'abfss://landing@<youraccount>.dfs.core.windows.net/';
```

Expected Output

An empty listing (no error) confirms Databricks can read the container even before any files are in it.

Common Errors

- **403 / Forbidden** — the IAM role assignment on the storage account hasn't propagated yet (can take a few minutes) or was assigned to the wrong identity.
- **External location already exists** — names must be unique per metastore; pick a different name.
- **Hierarchical namespace not enabled** — you can't retrofit this after creation; delete and recreate the storage account with it enabled from the start.

Cleanup

Nothing to clean up yet — you'll use this landing container starting Task 6.

Mini Exercise

Manually upload any small file to the `landing` container via the Azure Portal's Storage Browser, then re-run the `LIST` command above and confirm it shows up.

TASK 6 — Ingestion Through Staging

Difficulty: Medium

What You'll Learn

- The difference between source, landing, staging, and bronze
- Reading multiple files from one folder at once

Architecture

```
orders_001.csv, orders_002.csv, orders_003.csv
      | (uploaded to landing/orders/)
      v
    spark.read.csv(folder path)
      |
      v
  bronze.orders_v2 (all 3 files combined)
```

Why

Source is wherever data originates (an app database, in our case a CSV). **Landing** is the raw drop zone in cloud storage, untouched. **Staging** is a transient area for validation before committing to Bronze. **Bronze** is the permanent, queryable raw record. Keeping these separate means a bad landing file never corrupts your permanent Bronze history.

Cost Warning

Same compute; uploading three small CSVs to Storage costs a negligible amount.

Setup

- 1 Save the two CSV blocks below as `orders_002.csv` and `orders_003.csv` (you already have the original 15 rows as "orders_001" conceptually from Task 2).
- 2 In the Azure Portal Storage Browser (or Databricks Catalog Explorer's External Locations browser), create a folder `orders/` inside the `landing` container and upload all three files there.

orders_002.csv

```
order_id,customer_id,product_id,quantity,order_date,status
5016,1005,2003,1,2025-04-10,COMPLETED
5017,1006,2009,2,2025-04-10,COMPLETED
5018,1007,2002,1,2025-04-11,COMPLETED
5019,1008,2006,1,2025-04-11,COMPLETED
5020,1009,2001,3,2025-04-12,COMPLETED
```

orders_003.csv

```
order_id,customer_id,product_id,quantity,order_date,status
5021,1010,2005,1,2025-04-13,COMPLETED
5022,1001,2008,1,2025-04-13,COMPLETED
5023,1002,2009,2,2025-04-14,COMPLETED
5024,1003,2010,1,2025-04-14,COMPLETED
5025,1004,2001,,2025-04-15,COMPLETED
```

Cell 1 — read the whole folder at once

```
orders_folder = "abfss://landing@<youraccount>.dfs.core.windows.net/orders/"
```

```
orders_all = spark.read.csv(orders_folder, header=True, inferSchema=True)
print(orders_all.count()) # should include rows from all 3 files
orders_all.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.bronze.orders_v2")
```

Expected Output

Row count reflects the combined rows from all three CSVs (28, including the intentionally blank quantity in `orders_003` for later tasks).

Verify

```
%sql SELECT COUNT(DISTINCT order_id) FROM ecommerce_dev.bronze.orders_v2;
```

Common Errors

- **Schema mismatch between files** — if a later file has an extra/missing column, add `mergeSchema=True` or fix the source file.
- **Only some files read** — check the folder path ends with a trailing slash and actually contains all three files.

Cleanup

Keep the folder — Task 7 reads from this exact location.

Mini Exercise

Add a fourth intentionally malformed CSV (e.g., a row missing a comma) to the folder, re-run the read, and observe what Spark does with it by default.

TASK 7 — Auto Loader

Difficulty: Medium-Hard

What You'll Learn

- `spark.read` vs `spark.readStream` vs the `cloudFiles` source
- Incremental ingestion, checkpoints, schema location, new-file detection

Architecture

```
landing/orders/*.csv
  |
  v
spark.readStream.format("cloudFiles")  (Auto Loader)
  |  tracks which files it has already seen via the checkpoint
  v
bronze.orders_autoloader  (append-only, incremental)
```

Why

`spark.read` re-reads everything, every time — fine for Task 6's tiny folder, wasteful at real scale. `cloudFiles` (Auto Loader) remembers which files it already processed via a checkpoint, so re-running only picks up what's new.

Cost Warning

Use `trigger(availableNow=True)` — this processes everything currently available and then stops, instead of leaving a stream running (and billing) continuously. Never leave a Structured Streaming query running unattended in this learning project.

Cell 1 — Auto Loader batch-style run

```
checkpoint_path = "/Volumes/workspace/default/raw_files/_checkpoints/orders_autoloader"
schema_location = "/Volumes/workspace/default/raw_files/_schemas/orders_autoloader"

stream_df = (
    spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.schemaLocation", schema_location)
        .option("header", "true")
        .load("abfss://landing@<youraccount>.dfs.core.windows.net/orders/")
)

query = (
    stream_df.writeStream
        .format("delta")
        .option("checkpointLocation", checkpoint_path)
        .trigger(availableNow=True)
        .toTable("ecommerce_dev.bronze.orders_autoloader")
)
query.awaitTermination()
```

Expected Output

The cell finishes (doesn't hang) because of `availableNow=True`. The new table has the same row count as Task 6's combined read.

Verify

Re-run the same cell with no new files in the folder — row count in the table shouldn't change, proving Auto Loader skipped already-seen files.

Common Errors

- **Stream never finishes** — you forgot `trigger(availableNow=True)`; it's running continuously (this costs money — stop it immediately from the notebook's running-cells indicator).
- **Schema inference failed** — Auto Loader needs to sample files first; re-run once after the schema location has been populated.

Cleanup

If you ever start a stream without `availableNow` by mistake, click the cancel/stop icon on the running cell immediately.

Mini Exercise

Upload one more small CSV to the landing/orders folder, re-run Cell 1, and confirm only the new file's rows get added.

TASK 8 — Incremental ETL

Difficulty: Hard

What You'll Learn

- Idempotency: re-running a pipeline shouldn't duplicate data
- MERGE INTO for upserts
- Failure recovery basics

Architecture

```
bronze.orders_autoloader (append-only, from Task 7)
    |
    v
MERGE INTO silver.orders_clean
ON orders_clean.order_id = source.order_id
WHEN MATCHED -> update, WHEN NOT MATCHED -> insert
```

Why

Auto Loader guarantees each file is only ingested once into Bronze, but Bronze-to-Silver still needs its own idempotency: if you re-run the Silver step twice, you must not get duplicate rows. MERGE (upsert) is the standard fix.

Cost Warning

No new resources; same compute.

Cell 1 — idempotent Silver merge

```
from delta.tables import DeltaTable
from pyspark.sql.functions import col, to_date

bronze = spark.table("ecommerce_dev.bronze.orders_autoloader") \
    .filter(col("status") == "COMPLETED") \
    .filter(col("quantity").isNotNull()) \
    .withColumn("order_date", to_date(col("order_date")))

if not spark.catalog.tableExists("ecommerce_dev.silver.orders_incremental"):
    bronze.write.format("delta").saveAsTable("ecommerce_dev.silver.orders_incremental")
else:
    target = DeltaTable.forName(spark, "ecommerce_dev.silver.orders_incremental")
    (target.alias("t")
     .merge(bronze.alias("s"), "t.order_id = s.order_id")
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute())
```

Expected Output

First run creates the table. Re-running the exact same cell with no new Bronze data leaves the row count unchanged (no duplicates).

Verify

Run `SELECT order_id, COUNT(*) FROM ecommerce_dev.silver.orders_incremental GROUP BY order_id HAVING COUNT(*) > 1;` — should return zero rows.

Common Errors

- **Duplicate rows appear** — the merge condition key isn't actually unique in the source; check for true duplicate `order_ids` in Bronze first.

- **whenMatchedUpdateAll fails on schema mismatch** — source and target columns must line up; check both schemas with `printSchema()`.

Cleanup

Nothing to delete.

Mini Exercise

Manually re-run Task 7's Auto Loader cell, then this task's merge cell, twice in a row, and confirm the final row count is stable both times.

TASK 9 — Complete ETL Pipeline

Difficulty: Hard

What You'll Learn

- Chaining everything built so far into one coherent set of notebooks
- Structuring notebooks so a Job (Task 10) can call them independently

Architecture

```
landing/orders/*.csv
|
v (Notebook A: Bronze_Ingest)
bronze.orders_autoloader
|
v (Notebook B: Silver_Transform)
silver.orders_incremental
|
v (Notebook C: Gold_Aggregate)
gold.daily_sales, gold.customer_summary, gold.product_performance
```

Why

Tasks 6-8 worked but lived in one exploratory notebook. Before orchestrating anything (Task 10), split the logic into three focused notebooks — this is what a Lakeflow Job actually calls, task by task.

Cost Warning

Same compute, run sequentially, one after another — no parallel resources needed yet.

Setup

- 1 Create three notebooks: Bronze_Ingest, Silver_Transform, Gold_Aggregate.
- 2 Bronze_Ingest: paste in Task 7's Cell 1 code.
- 3 Silver_Transform: paste in Task 8's Cell 1 code.
- 4 Gold_Aggregate: paste in Task 4's Cell 2-3 code, but read from `ecommerce_dev.silver.orders_incremental` instead of `orders_clean`.
- 5 Run all three top to bottom, in order, once, to confirm each works standalone.

Expected Output

All three notebooks run without errors when executed independently, in sequence.

Verify

Query all three Gold tables and confirm the numbers match what you'd expect from the current Bronze/Silver row counts.

Common Errors

- **Notebook B fails when run alone** — it depends on state from Notebook A having already run (the Bronze table must exist first) — this is expected, and exactly why Task 10 adds explicit dependencies.

Cleanup

Nothing to delete — these three notebooks are what Task 10 turns into a Job.

Mini Exercise

Time roughly how long each notebook takes to run end-to-end — you'll use this to sanity-check the Job's run duration in Task 10.

TASK 10 — Lakeflow Jobs

Difficulty: Hard

What You'll Learn

- Job, Task, dependency ("Depends on"), parameters, compute, retries, timeout
- Reading logs and run history

Architecture

```
Job: ecommerce_etl_pipeline
  Task 1: bronze_ingest    (runs Bronze_Ingest notebook)
  Task 2: silver_transform (runs Silver_Transform, Depends on: bronze_ingest)
  Task 3: gold_aggregate   (runs Gold_Aggregate,  Depends on: silver_transform)
```

Why

A **Job** orchestrates multiple **Tasks** in the correct order automatically, instead of you clicking "Run" on three notebooks by hand. This is the real building block behind Task 11's schedule and Task 12's event trigger.

Cost Warning

If you choose serverless for the job's tasks, it bills only for actual run time. If you attach a classic job cluster, it starts fresh for the run and terminates afterward automatically (unlike an all-purpose cluster, a job cluster doesn't need manual auto-termination) — still verify it shuts down in the Compute tab after the run.

UI Steps

- 1 Left sidebar → **Jobs & Pipelines** → **Create** → **Job**.
- 2 Name it `ecommerce_etl_pipeline`.
- 3 First task: Task name `bronze_ingest`, Type **Notebook**, Path → browse to `Bronze_Ingest`, Compute → Serverless (or a job cluster with the smallest node type).
- 4 Click **+ Add task** → name `silver_transform`, Type **Notebook**, Path → `Silver_Transform`, and in **Depends on** select `bronze_ingest`.
- 5 Add a third task `gold_aggregate`, Path → `Gold_Aggregate`, Depends on → `silver_transform`.
- 6 On each task, expand advanced settings if you want to set **Retries** (e.g. 1) and a **Timeout** (e.g. 15 min) — sensible defaults for a small learning job.
- 7 Click **Run now** at the top to trigger a manual run.

Expected Output

The job's run view shows a small DAG with three boxes connected by arrows, each turning green in sequence as it completes.

Verify

Jobs & Pipelines → your job → **Run history** tab → click the latest run → confirm all three tasks show Succeeded, and open each task's logs.

Common Errors

- **Task fails: table not found** — an upstream task didn't actually finish before this one ran; check the Depends on setting is correctly configured.

- **Job stuck in Pending** — compute is still provisioning; wait a couple of minutes before assuming it's broken.

Cleanup

Job clusters terminate automatically after the run — confirm in the Compute tab that nothing is left running.

Mini Exercise

Deliberately break Task 2 (typo a column name in Silver_Transform), run the job, and read the failure in the task's logs before fixing it.

TASK 11 — Scheduled ETL

Difficulty: Hard

What You'll Learn

- Schedule, timezone, and how scheduling interacts with retries/timeout

Why

Scheduling is what turns a manually-triggered job into an actual pipeline that keeps your Gold tables fresh without you doing anything.

Cost Warning

A schedule means the job runs unattended, repeatedly. If you leave a frequent schedule (e.g. every 5 minutes) active on this learning project, it will keep consuming compute indefinitely — pause it as soon as you've confirmed it works.

UI Steps

- 1 Open your job → Job details pane (right side) → **Schedules & Triggers** → **Add trigger**.
- 2 Trigger type: **Scheduled**. Set it to run daily at a time of your choosing, and pick your timezone (e.g. Asia/Kolkata).
- 3 Click **Save**.
- 4 Immediately after confirming it appears correctly, click **Pause** on the trigger so it doesn't keep firing while you move on to later tasks.

Expected Output

The Schedules & Triggers section shows your schedule with a Paused state.

Verify

Toggle it to active briefly, confirm a run kicks off automatically at (or shortly after) the scheduled time, then pause it again.

Common Errors

- **Wrong timezone** — double check Asia/Kolkata vs UTC; a schedule set in UTC will fire 5.5 hours off from what you expect.

Cleanup

Leave the trigger Paused until Task 16, so it isn't silently running in the background.

Mini Exercise

Explain in your own words (a sentence is enough) when a daily schedule is the wrong choice compared to Task 12's event-driven trigger.

TASK 12 — Event-Driven ETL

Difficulty: Very Hard

What You'll Learn

- Event-driven architecture concepts: publisher, event, subscriber/consumer, trigger, event source
- Databricks' native file-arrival trigger

Architecture

```
New file lands in landing/orders/ (event source / publisher)
      |
      v
File Arrival Trigger (subscriber, polls ~ every 1 min)
      |
      v
ecommerce_etl_pipeline job starts automatically
```

Why

A schedule runs whether or not there's new data. An **event-driven** pipeline reacts only when something actually changed — here, a new file landing in your storage. The **publisher** is whatever writes the file; the **event** is the file's arrival; the Databricks job is the **subscriber/consumer** reacting to it.

Before Starting

Requires Unity Catalog (you have it) and the storage location from Task 5 to be a Unity Catalog external location or volume (it is — `ecommerce_landing`).

Cost Warning

Databricks' native file arrival trigger polls for new files roughly every minute at no extra Databricks cost beyond the normal cost of listing files in your storage account — this is the cheapest correct way to do this, so we use it instead of building a separate Azure Event Grid + Function pipeline (that's the production alternative — learn it later).

UI Steps

- 1 Open your job → **Schedules & Triggers** → **Add trigger**.
- 2 Trigger type: **File arrival**.
- 3 Storage location: the URL or Volume/External Location path from Task 5, e.g. `abfss://landing@<youraccount>.dfs.core.windows.net/orders/`.
- 4 Save.

Expected Output

The trigger is listed as active in Schedules & Triggers.

Verify

Upload a brand-new CSV to the landing/orders folder and, within about a minute, confirm a new job run appears automatically in Run history — without you clicking Run now.

Common Errors

- **Trigger never fires** — the storage location isn't registered as a Unity Catalog external location/volume, or you lack READ permission on it.

- **Overwriting an existing file doesn't trigger** — expected behavior; only genuinely new files trigger a run.

Cleanup

Leave this trigger active only while testing; pause it before moving to Task 13 so test uploads don't keep firing runs unexpectedly.

Mini Exercise

In one or two sentences, explain why a file-arrival trigger is a better fit than a tight 1-minute schedule for this use case, even though both would react quickly.

TASK 13 — Notifications

Difficulty: Very Hard

What You'll Learn

- Job-level notifications: on Start, Success, Failure, Duration warning

Why

A pipeline that fails silently is worse than no pipeline — you find out days later when someone asks why Gold tables are stale. Notifications close that gap.

Cost Warning

Free — email notifications don't consume compute or DBUs.

UI Steps

- 1 Open your job → Job details pane → **Notifications** → **Add notification**.
- 2 Destination: **Email address** → enter your own email.
- 3 Check the boxes for **Success** and **Failure** (leave Start unchecked to avoid noise for a small learning job).
- 4 Save.

Expected Output

The Notifications section lists your email against Success and Failure events.

Verify

Trigger a manual run (Run now) and confirm you receive a success email; then deliberately break a task and confirm you receive a failure email on the next run.

Common Errors

- **No email arrives** — check spam folder first; also note job-level notifications aren't re-sent for automatic retries of the same failed task — that's expected, not a bug.

Cleanup

Nothing to clean up.

Mini Exercise

Add a second notification destination (a different email or your phone's email-to-SMS address if you have one) and confirm both receive the alert.

TASK 14 — Data Quality

Difficulty: Very Hard

What You'll Learn

- Writing explicit data quality checks between Silver and Gold
- Quarantining bad rows instead of silently dropping or silently passing them

Architecture

```
silver.orders_incremental
  |
  v
Data Quality checks (new notebook: DataQuality_Check)
  |           |
passes       fails
  |           |
  v           v
gold.* (Task 4/9 aggregation, now reading only validated rows)
      silver.orders_quarantine
```

Why

Silently letting bad rows (null IDs, negative amounts, duplicate order_ids) flow into Gold corrupts every business number downstream. Explicit checks with a quarantine table make bad data visible instead of invisible.

Cost Warning

No new resources; same compute.

Cell 1 — DataQuality_Check notebook

```
from pyspark.sql.functions import col
```

```
silver = spark.table("ecommerce_dev.silver.orders_incremental")
```

```
rules = (
    col("order_id").isNotNull()
    & col("customer_id").isNotNull()
    & (col("quantity") >= 0)
)
```

```
good = silver.filter(rules)
bad  = silver.filter(~rules)
```

```
good.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.silver.orders_validated")
bad.write.format("delta").mode("overwrite").saveAsTable("ecommerce_dev.silver.orders_quarantine")
```

```
bad_count = bad.count()
print(f"Quarantined rows: {bad_count}")
```

```
if bad_count > 0:
    # visible in job logs; doesn't hard-fail the job for a learning project,
    # but you could `raise Exception(...)` here to force the job to fail loudly instead
    print("WARNING: some rows failed data quality checks – see orders_quarantine")
```

Expected Output

orders_003's row with a blank quantity (from Task 6) lands in orders_quarantine; everything else lands in orders_validated.

Verify

```
SELECT * FROM ecommerce_dev.silver.orders_quarantine; should show exactly the intentionally bad row(s).
```

Common Errors

- **Every row lands in quarantine** — a rule is evaluating against the wrong column name or type; double-check with `printSchema()`.
- **duplicate order_id not allowed** — add a rule using a window function count or an anti-join against already-seen IDs, rather than `dropDuplicates` (which would silently keep one copy instead of quarantining both).

Cleanup

Nothing to delete — wire this notebook into the job as a 4th task, between `silver_transform` and `gold_aggregate`, updating `gold_aggregate` to read from `orders_validated`.

Mini Exercise

Add one more rule: quantity must be less than 100 (catch obviously wrong data entry), and confirm it doesn't accidentally quarantine any of your legitimate rows.

TASK 15 — Failure Handling

Difficulty: Very Hard

What You'll Learn

- Reading task logs to identify the real cause of a failure
- Using Repair run to re-run only the failed tasks
- Avoiding duplicate processing on retry

Why

Pipelines fail in production constantly — corrupt files, schema drift, transient cloud errors. The skill that matters isn't avoiding failure, it's debugging and recovering from it quickly.

Cost Warning

No new resources; re-running a failed job task costs the same as any normal run of that task.

Exercises to run yourself

- 1 **Corrupt file:** upload a CSV to landing/orders with a broken header row. Run the job. Open the failed task's logs — read the actual Spark error message, not just "task failed". Fix by removing the bad file, then use **Repair run** (on the job's run page) to re-run only the failed task instead of the whole job.
- 2 **Missing column:** upload a CSV missing the status column. Observe whether Auto Loader's schema evolution handles it or fails — read the exact exception.
- 3 **Invalid data type:** put a text value in the quantity column. Confirm the value comes through as null after casting rather than crashing the whole job (design choice from Task 2), and gets caught by Task 14's quality check.
- 4 **Duplicate data:** re-upload orders_001.csv unchanged. Confirm Task 8's MERGE logic prevents duplicate rows in Silver.

Verify

For each scenario, you should be able to state in one sentence: what broke, why, and how you confirmed the fix.

Common Errors

- **Repair run reprocesses everything** — you clicked Run now instead of Repair run; Repair run specifically only re-executes failed/skipped tasks.

Cleanup

Remove the intentionally broken test files from the landing folder once you're done, so they don't linger and confuse Task 16's final run.

Mini Exercise

Write a short paragraph (for your own notes / future README) summarizing the most surprising failure mode you hit here.

TASK 16 — Final Production-Style Project

Difficulty: Very Hard

What You'll Learn

- Pulling every previous task into one coherent, documented project

Architecture

```
SOURCE -> AZURE STORAGE (landing) -> AUTO LOADER -> BRONZE
      -> SILVER -> DATA QUALITY -> GOLD -> LAKEFLOW JOB
      |
      SCHEDULE + FILE-ARRIVAL TRIGGER
      |
      SUCCESS/FAILURE -> NOTIFICATION
```

Final Checklist

- Job has all tasks wired in order: bronze_ingest -> silver_transform -> data_quality_check -> gold_aggregate.
- File arrival trigger active on landing/orders/, daily schedule present as a fallback (both can coexist).
- Notifications configured on Success and Failure.
- All classic clusters (if any) have auto-termination set; nothing is left running unattended.
- Azure Cost Management checked once more, filtered to your resource group, to confirm no surprise spend.

Deliverables to Produce Yourself

- 1 **Architecture diagram** — the block diagram above, redrawn with your actual resource names.
- 2 **Project folder structure** — list your notebooks (Bronze_Ingest, Silver_Transform, DataQuality_Check, Gold_Aggregate) and their workspace paths.
- 3 **Catalog/schema/table structure** — `ecommerce_dev.bronze/silver/gold` with every table listed.
- 4 **ETL workflow diagram** — export or screenshot the job's DAG view from Jobs & Pipelines.
- 5 **Cost-control checklist** — the bullet list from Section 7 of your setup guide, filled in with what you actually configured.
- 6 **Troubleshooting guide** — the Common Errors sections from every task above, consolidated.
- 7 **GitHub README** — summarize scenario, architecture, tech stack, and how to reproduce it.
- 8 **Resume bullet** — one or two lines describing the project, its scale, and the concepts demonstrated.
- 9 **Interview questions** — write 5-10 questions you'd expect to be asked about this project (e.g. "why Auto Loader over spark.read?", "how does your pipeline stay idempotent?").
- 10 **Concepts learned list** — pull the "What You'll Learn" bullets from every task in this document into one list.

What's Next

- Once this is solid, natural next steps include: Databricks Asset Bundles for CI/CD, dbt on Databricks SQL, streaming with true continuous triggers, and the Databricks Certified Data Engineer Associate certification.